Software Requirements Specification: ProveIt Platform

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for the ProveIt platform. It is based on the current codebase across:

- frontend: React user application
- backend: Express/MongoDB API
- admin: Angular admin dashboard

1.2 Product Scope

ProveIt is a technical interview and collaborative coding platform. It supports:

- Public marketing, pricing, blog, about, and contact pages
- User authentication through Clerk
- Coding practice problems with Judge0 execution
- Score, solved problem tracking, levels, and leaderboard
- Live coding sessions with Stream video/chat
- Scheduling through weekly availability and invitations
- Notifications and email for session/invitation events
- AI interview practice with CV analysis, voice-oriented question flow, scoring, and results
- Subscription plan management
- Admin dashboard for users, sessions, AI interviews, analytics, and blog posts

1.3 Intended Users

User Type	Description
Guest	Can view public pages and blog posts.
Authenticated user	Can practice problems, create/join sessions, schedule invitations, manage availability, use AI interviews, and manage subscription.
Admin	Can view platform analytics and manage users, sessions, AI interviews, and blog posts.

2. Overall Description

2.1 Product Perspective

The platform is composed of:

- React SPA served separately in development and statically by ba
- Express REST API with Clerk middleware.
- MongoDB persistence through Mongoose.
- Angular admin dashboard using Clerk-backed admin auth.
- Third-party integrations:

- Clerk for authentication
- Stream for video and chat
- Judge0 for code execution
- Resend for email
- Inngest for background AI interview completion events
- ITI/OpenAI-compatible gateway for CV/interview AI features

2.2 Operating Environment

- Node.js: >=20.18.0 for backend/admin
- Backend: Express 5
- Database: MongoDB
- Frontend: React 19, Vite, Tailwind CSS, DaisyUI
- Admin: Angular 17, Tailwind CSS, DaisyUI

3. Functional Requirements

FR-1 Public Website

ID	Requirement
FR-1.1	The system shall display a public home page without authentication.
FR-1.2	The home page shall support English and Arabic translations.
FR-1.3	The public pages shall include pricing, about, contact, blog listing, and blog detail pages.
FR-1.4	The layout shall support LTR and RTL modes.
FR-1.5	The site footer shall render once per page except hidden workspace routes.

FR-2 Authentication and User Identity

ID	Requirement
FR-2.1	The system shall use Clerk for user authentication.
FR-2.2	The backend shall map Clerk users to MongoDB User records.
FR-2.3	Protected API routes shall require an authenticated user.
FR-2.4	Admin API routes shall require an authenticated admin user.

FR-3 Problems and Code Execution

ID	Requirement
FR-3.1	The system shall display practice problems from frontend problem data.
FR-3.2	Problems shall be grouped by difficulty: Easy, Medium, Hard.
FR-3.3	Users shall be able to open a problem and write code.
FR-3.4	The backend shall execute supported languages through Judge0.

FR-3.5	Supported execution languages shall include JavaScript, Python, and Java.
FR-3.6	The system shall compare output against expected output when provided.
FR-3.7	The system shall store submission attempts and score changes.

FR-4 Scoring, Levels, and Leaderboard

ID	Requirement
FR-4.1	The system shall award points after accepted submissions.
FR-4.2	Easy problems shall award 10 solved points.
FR-4.3	Medium problems shall award 20 solved points.
FR-4.4	Hard problems shall award 35 solved points.
FR-4.5	A first accepted attempt shall award a 5 point bonus.
FR-4.6	A daily activity bonus of 3 points may be awarded once per day.
FR-4.7	A wrong submission penalty of -2 may be applied once per problem.
FR-4.8	User score shall never be below 0.
FR-4.9	User level shall be derived from total score.
FR-4.10	The dashboard shall show the user's score, level, rank, and solved problems.
FR-4.11	The leaderboard shall rank users by score descending.
FR-4.12	Users with identical scores shall share the same rank.

FR-5 Live Coding Sessions

ID	Requirement
FR-5.1	Authenticated users shall be able to create sessions by selecting difficulty.
FR-5.2	A session shall create a Stream video call and chat channel.
FR-5.3	A session shall store host, participant, selected difficulty, selected problem, status, and call id.
FR-5.4	Session participants shall choose a problem from the selected difficulty inside the session.
FR-5.5	The selected problem shall be persisted on the session and visible in dashboard/session lists.
FR-5.6	A host shall be able to end a session.
FR-5.7	Completed sessions shall no longer be joinable as active sessions.
FR-5.8	Reserved invitation sessions shall only be joinable by the invited participant.

FR-6 Availability and Scheduling

ID	Requirement
FR-6.1	Authenticated users shall be able to define weekly availability.
FR-6.2	Availability shall be stored as day-based time slots in Africa/Cairo time.
FR-6.3	Users shall be searchable for scheduling.
FR-6.4	Search results shall support filtering by user level.
FR-6.5	Users shall be able to send invitations only for available recipient time slots.
FR-6.6	Scheduled invitations shall store date, time, duration, difficulty, optional message, and status.
FR-6.7	Accepted invitations shall create a full session entity.

FR-7 Invitations, Notifications, and Email

ID	Requirement
FR-7.1	The system shall create in-app notifications for new invitations.
FR-7.2	The system shall send invitation emails when configured with Resend.
FR-7.3	The invited user shall accept or decline invitations.
FR-7.4	Accepted invitations shall notify the sender and create a session link.
FR-7.5	Declined invitations shall notify the sender.
FR-7.6	Starting an invitation-created session shall notify the other session member in-app and by email.
FR-7.7	Users shall be able to mark notifications as read.
FR-7.8	Users shall be able to mark all notifications as read.
FR-7.9	Users shall be able to delete notifications.

FR-8 AI Interview

ID	Requirement
FR-8.1	Users shall upload a PDF CV.
FR-8.2	The backend shall reject non-PDF CV uploads.
FR-8.3	The backend shall extract text from the PDF or use image fallback for scanned PDFs.
FR-8.4	The AI service shall create or update a candidate profile from the CV.
FR-8.5	A user shall not start an AI interview without a candidate profile unless mock profile mode is used.
FR-8.6	AI interviews shall support English and Arabic.
FR-8.7	AI interviews shall generate at least 5 questions.
FR-8.8	Users shall submit answers, and the backend shall store feedback, score, time spent, and follow-up state.

FR-8.9	The system shall generate final results and mark the interview completed.
FR-8.10	Completed AI interviews shall be available in history.

FR-9 Subscription

ID	Requirement
FR-9.1	Users shall have a subscription record created on demand.
FR-9.2	Free plan shall allow 3 AI interviews per month.
FR-9.3	Pro and Premium plans shall be treated as unlimited for AI interviews.
FR-9.4	The system shall simulate successful payments for Visa/Mastercard-like card numbers.
FR-9.5	The system shall simulate declines for configured card patterns.
FR-9.6	Users shall be able to cancel and return to Free plan.
FR-9.7	Subscription confirmation email shall be attempted when payment succeeds.

FR-10 Blog

ID	Requirement
FR-10.1	Guests and authenticated users shall be able to view published blog posts.
FR-10.2	Blog listing shall fetch published posts from the backend.
FR-10.3	Blog detail shall load a post by slug.
FR-10.4	Blog content shall support localized English and Arabic fields.
FR-10.5	Admins shall be able to create, edit, publish/unpublish, and delete blog posts.

FR-11 Admin Dashboard

ID	Requirement
FR-11.1	Admin users shall access the Angular admin dashboard.
FR-11.2	Admin dashboard shall show totals for users, sessions, AI interviews, active subscriptions, pending invitations, and subscription distribution.
FR-11.3	Admin dashboard shall show session and AI interview charts.
FR-11.4	Admins shall view and paginate users.
FR-11.5	Admins shall update user roles.
FR-11.6	Admins shall view sessions and AI interviews.
FR-11.7	Admins shall view top candidates and problem/category stats.
FR-11.8	Admins shall manage blog posts.

4. Non-Functional Requirements

NFR-1 Security

- Protected API routes shall require Clerk authentication.
- Admin API routes shall require admin authorization.
- CORS shall be restricted to configured client/admin origins.
- Sensitive keys shall be stored in environment variables.
- Email failures shall not crash core invitation/subscription flows.

NFR-2 Performance

- Frontend routes shall be lazy-loaded.
- Heavy home page visual modules shall be lazy-loaded.
- Backend list endpoints shall use pagination where implemented.
- MongoDB indexes shall support common user/session/invitation queries.

NFR-3 Reliability

- Backend global error handler shall prevent leaking stack traces to clients.
- Session end shall mark MongoDB session completed before external Stream cleanup.
- AI result generation shall be idempotent for completed sessions.
- Email helpers shall no-op safely when Resend config is missing.

NFR-4 Usability

- The UI shall support responsive mobile, tablet, and desktop layouts.
- The UI shall support English and Arabic with RTL document direction.
- Code sample/editor areas shall remain LTR even in Arabic mode.
- Public pages shall not require authentication.

NFR-5 Maintainability

- Backend routes shall be organized by domain.
- Database models shall remain typed through Mongoose schemas.
- Frontend API calls shall be grouped in frontend/src/api.
- Admin API calls shall be grouped in Angular admin services.

5. External Interface Requirements

5.1 REST API Base Paths

Base Path	Purpose	Auth
/api/users	Score, leaderboard, solved problems	Required
/api/subscription	Subscription plan and limit checks	Required
/api/sessions	Live coding sessions	Required
/api/notifications	In-app notifications	Required
/api/invitations	Scheduling and invitation flow	Required
/api/code	Judge0 execution	Required

/api/chat	Stream chat token	Required
/api/availability	Weekly availability	Required
/api/ai-interview	CV upload and AI interviews	Required
/api/blog	Public blog listing/detail	Public
/api/admin	Admin dashboard and blog management	Admin

5.2 Public Frontend Routes

Route	Description
/	Home page
/pricing	Pricing
/about	About
/blog	Blog listing
/blog/:slug	Blog detail
/contact	Contact

5.3 Authenticated Frontend Routes

Route	Description
/dashboard	User dashboard
/problems	Problem list
/problem/:id	Problem workspace
/session/:id	Live session workspace
/availability/settings	Availability settings
/schedule	Schedule invitations
/invitations	Invitation center
/subscription	Subscription management
/ai-interview	AI interview hub
/ai-interview/upload-cv	CV upload
/ai-interview/session/:sessionId	AI interview session
/ai-interview/results/:sessionId	AI interview results

5.4 Admin Routes

Route	Description
/login	Admin login
/dashboard	Admin dashboard

/users	User management
/sessions	Session management
/ai-interviews	AI interview management
/blog-posts	Blog post management

6. Data Requirements

- User identity shall be synchronized with Clerk.
- Session records shall remain durable in MongoDB even when Str
- Accepted problem submissions shall preserve the first accepted s
- Blog posts shall store localized EN/AR content.
- AI interview sessions shall preserve question-level evidence for fi

7. Assumptions and Constraints

- Practice problems are currently stored as frontend static data, not
- Payment is simulated, not integrated with a real payment provider
- AI interview paywall code is present but disabled in aiInterviewCo
- Email requires RESEND_API_KEY and RESEND_FROM; otherw
- Stream requires STREAM_API_KEY and STREAM_API_SECRE
- Judge0 may use RapidAPI or fall back to public CE endpoint base